

NiceGUI @app.on_page_exception 全面详细解析

`@app.on_page_exception` 是 NiceGUI 2.20.0 版本新增的装饰器，用于定义**自定义页面异常处理逻辑**，核心作用是替换框架默认的“sad face”错误页面，让开发者可以针对不同类型的页面异常（如超时、运行时错误）定制可视化的错误提示，同时支持精准过滤异常类型、控制是否回退到默认处理逻辑。

一、核心定位与设计目标

1. 核心价值

- 替换默认错误页：覆盖框架内置的通用错误页面，打造符合业务风格的异常展示；
- 精准处理异常：支持仅拦截特定类型的异常（如 `TimeoutError`），其他异常仍使用默认逻辑；
- 灵活定制内容：可通过 NiceGUI 组件自由设计错误页的 UI（图标、文本、堆栈信息等）；
- 安全可控：可选择性展示异常详情（生产环境建议隐藏堆栈，避免泄露敏感信息）。

2. 关键设计规则

- 同步函数要求：处理函数必须是**同步函数**（不能用 `async def`），且逻辑与普通页面函数一致（通过 `ui.xxx` 组件构建页面）；
- 异常透传机制：若在处理函数中重新抛出异常（`raise exception`），框架会回退到默认错误页；
- 参数可选性：处理函数可接收 `exception` 参数（获取异常对象），也可省略（仅处理所有页面异常）；
- 生效范围：仅拦截**页面路由函数**（`@ui.page` 装饰的函数）执行过程中抛出的未捕获异常，不影响全局异常（`app.on_exception` 负责）。

二、核心用法与语法规则

1. 基础语法

```
from nicegui import app, ui

# 装饰器注册自定义页面异常处理函数
@app.on_page_exception
def custom_error_page(exception: Exception) -> None:
    # 1. 可选：过滤异常类型（仅处理特定异常）
    if not isinstance(exception, 目标异常类型):
        raise exception # 非目标异常，回退到默认错误页
    # 2. 构建自定义错误页 UI（与普通页面函数写法一致）
    with ui.column().classes('样式类'):
        ui.icon('错误图标')
        ui.label(f'异常信息: {exception}')
```

2. 关键参数与返回值

元素	说明
<code>exception:</code> <code>Exception</code>	可选参数，接收触发页面异常的原始异常对象，可通过 <code>isinstance</code> 判断类型、 <code>str(exception)</code> 获取异常信息
返回值	必须为 <code>None</code> ，处理函数仅负责构建 UI，无需返回任何内容
重新抛出异常	<code>raise exception</code> 会终止自定义处理，框架自动使用默认错误页

三、完整示例拆解（用户提供的 main.py）

以下基于用户示例代码，逐行解析核心逻辑与效果：

1. 代码全量解析

```
import traceback # 用于获取异常堆栈信息
from nicegui import app, ui

# 注册自定义页面异常处理函数
@app.on_page_exception
def timeout_error_page(exception: Exception) -> None:
    # 步骤1: 过滤异常类型--仅处理 TimeoutError
    if not isinstance(exception, TimeoutError):
        raise exception # 非超时异常，回退到默认错误页
    # 步骤2: 构建自定义错误页 UI
    with ui.column().classes('absolute-center items-center gap-8'): # 居中布局，间距8
        ui.icon('sym_o_timer', size='x1') # 显示定时器图标（超大号）
        ui.label(f'{exception}').classes('text-2x1') # 显示异常信息（2号大文本）
        ui.code(traceback.format_exc(chain=False)) # 显示异常堆栈（仅堆栈，无链式异常）

# 测试页面1: 触发 TimeoutError（会走自定义错误页）
@ui.page('/raise_timeout_error')
def raise_timeout_error():
    raise TimeoutError('This took too long') # 抛出超时异常

# 测试页面2: 触发 RuntimeError（会走默认错误页）
@ui.page('/raise_runtime_error')
def raise_runtime_error():
    raise RuntimeError('Something is wrong') # 抛出运行时异常

# 主页: 提供测试链接
@ui.page('/')
def page():
    ui.link('Raise timeout error (custom error page)', '/raise_timeout_error')
    ui.link('Raise runtime error (default error page)', '/raise_runtime_error')

ui.run()
```

2. 核心效果说明

访问路径	触发异常	处理逻辑	展示效果
/raise_timeout_error	TimeoutError	自定义处理函数拦截，不抛出异常	居中显示定时器图标 + 异常文本 + 堆栈代码块
/raise_runtime_error	RuntimeError	处理函数判断类型不匹配，raise exception 回退	框架默认的“sad face”错误页
/	无异常	正常执行页面函数	显示两个测试链接

3. 关键细节补充

- `traceback.format_exc(chain=False)`：仅获取当前异常的堆栈信息（`chain=False` 避免展示链式异常），生产环境建议删除该代码（防止泄露代码结构、路径等敏感信息）；
- 样式类说明：`absolute-center` 实现页面居中，`items-center` 让列内组件水平居中，`gap-8` 控制组件间距，`text-2x1` 是字体大小样式；
- 图标选择：`sym_o_timer` 是 NiceGUI 内置的定时器图标，可替换为 `error`、`warning` 等内置图标（参考 NiceGUI 图标文档）。

四、进阶用法与场景扩展

1. 处理所有页面异常（不过滤类型）

若需为所有页面异常定制统一的错误页，只需移除异常类型判断和重新抛出逻辑：

```
@app.on_page_exception
def all_error_page(exception: Exception) -> None:
    with ui.column().classes('absolute-center items-center gap-6'):
        ui.icon('error', size='4x1', color='red-500') # 红色错误图标
        ui.label('页面加载出错啦!').classes('text-3x1 font-bold')
        ui.label(f'错误详情: {str(exception)}').classes('text-lg text-gray-600')
        # 生产环境建议隐藏堆栈，仅开发环境展示
        # ui.code(traceback.format_exc())
        ui.button('返回首页', on_click=lambda: ui.navigate.to('/')).classes('mt-4')
```

2. 区分多种异常类型

通过多分支判断，为不同异常类型定制差异化的错误页：

```
@app.on_page_exception
def multi_error_page(exception: Exception) -> None:
    with ui.column().classes('absolute-center items-center gap-6'):
        if isinstance(exception, TimeoutError):
            ui.icon('sym_o_timer', size='4x1', color='orange-500')
            ui.label('请求超时').classes('text-3x1 font-bold')
        elif isinstance(exception, ValueError):
            ui.icon('edit', size='4x1', color='blue-500')
            ui.label('参数错误').classes('text-3x1 font-bold')
        else:
            raise exception # 其他异常回退到默认页
    ui.label(f'错误信息: {str(exception)}').classes('text-1g')
```

3. 无参数处理函数（简化版）

若无需获取异常对象，可省略 `exception` 参数，仅构建通用错误页：

```
@app.on_page_exception
def simple_error_page() -> None:
    with ui.column().classes('absolute-center items-center'):
        ui.label('页面加载失败，请稍后重试').classes('text-2x1')
        ui.button('刷新页面', on_click=lambda: ui.navigate.reload()).classes('mt-4')
```

五、与全局异常处理（`app.on_exception`）的对比

`@app.on_page_exception` 与 `app.on_exception` 是互补关系，核心区别如下：

维度	@app.on_page_exception	app.on_exception
拦截范围	仅页面路由函数（@ui.page）抛出的异常	应用内所有未捕获的异常（包括页面、后台任务、生命周期事件等）
处理目标	定制前端错误页 UI，面向用户	全局异常日志、告警，面向开发者 / 运维
函数类型	必须同步，且需构建 UI	支持同步 / 异步，无 UI 构建要求
异常透传	<code>raise exception</code> 回退到默认错误页	抛出异常会导致应用崩溃（需手动捕获）
核心用途	提升用户体验（友好的错误展示）	保障应用稳定性（异常监控、兜底）

组合使用示例

```
# 1. 页面异常：定制用户可见的错误页
@app.on_page_exception
def page_error(exception: Exception) -> None:
    if isinstance(exception, TimeoutError):
```

```

        with ui.column().classes('absolute-center'):
            ui.label('请求超时，请检查网络').classes('text-2xl')
    else:
        raise exception

# 2. 全局异常：记录日志并告警
def global_error(exception: Exception):
    print(f'全局异常: {type(exception).__name__} - {str(exception)}')
    # 示例：推送告警到钉钉/企业微信（生产环境逻辑）
    # send_alert(f'应用异常: {traceback.format_exc()}')
app.on_exception(global_error)

```

六、最佳实践与注意事项

1. 最佳实践

- 开发 / 生产环境区分：开发环境可展示堆栈信息（便于调试），生产环境仅展示友好提示（隐藏敏感信息）；

```

import os
@app.on_page_exception
def prod_error_page(exception: Exception) -> None:
    with ui.column().classes('absolute-center'):
        ui.label('页面出错了').classes('text-3xl')
        if os.getenv('ENV') == 'dev': # 仅开发环境展示堆栈
            ui.code(traceback.format_exc())

```

- 异常类型精准过滤：避免拦截所有异常，仅针对业务相关的异常（如超时、权限不足）定制，其他异常保留默认页；
- 提供操作入口：在错误页添加“返回首页”“刷新页面”等按钮，提升用户操作便利性；
- 结合日志：自定义错误页的同时，通过 `app.on_exception` 记录异常日志，便于排查问题。

2. 注意事项

- 同步函数限制：处理函数不能是异步（`async def`），否则框架会抛出错误；
- 路由跳转限制：错误页中避免触发复杂的路由跳转逻辑（如 `ui.navigate.to` 到需要权限的页面），防止二次异常；
- 样式隔离：建议使用 `absolute-center` 等布局类确保错误页全屏展示，避免受其他页面样式影响；
- 版本兼容性：仅在 NiceGUI 2.20.0 及以上版本可用，低版本需升级框架。

七、总结

`@app.on_page_exception` 是 NiceGUI 提升前端异常体验的核心工具，其核心优势在于：

1. 灵活的异常过滤：可精准拦截特定异常，不影响其他异常的默认处理；
2. 自由的 UI 定制：基于 NiceGUI 组件体系，可快速打造符合业务风格的错误页；
3. 安全的透传机制：通过重新抛出异常，保留框架默认的兜底能力；
4. 与全局异常处理互补：既面向用户展示友好提示，又面向开发者实现异常监控。

合理使用该装饰器，可显著降低用户面对页面错误时的困惑，同时兼顾开发调试的便利性和生产环境的安全性。